

Project 3 - 16-bit Processor (DOT Acceleration)

This document justifies the architecture of a single-cycle 16-bit processor, designed with a clear focus: to simplify decoding and maximise the operand space for executing dot product operations.

1. Instruction Set Architecture (ISA) and Machine Code

The architecture defines four fundamental instructions. The operation code (opcode) is strictly allocated to the 2 least significant bits (LSBs), ensuring a consistent decoding process across all operations. The table below outlines the structural format of the 16 bits alongside practical examples of assembly and machine code.

Instruction	16-bit Structure	Usage Example	Machine Code (Bin / Hex)
li rd, imm	imm[11] rd[3] 00	li R1, 5	00000000101 001 00 / 0x0014
add rd, rs1	0[8] rs1[3] rd[3] 01	add R2, R1	00000000 001 010 01 / 0x0029
dot rd, rs1	0[8] rs1[3] rd[3] 10	dot R0, R2	00000000 010 000 10 / 0x0042
dota rd, rs1, rs2	0[5] rs2[3] rs1[3] rd[3] 11	dota R0, R1, R3	00000 011 001 000 11 / 0x018B

2. Design Decisions Justification

The architecture reflects a pragmatic compromise between the available word space (16 bits) and the requirements of vector acceleration, based on the following criteria:

- ISA Expressiveness (Structural Trade-off):** Allocating only 2 bits for the opcode sets an absolute limit of 4 operations ($2^2 = 4$). By discarding additional selection logic (such as func3 and func7), we prioritised simplicity and capacity. This choice frees up crucial space for a highly expressive 11-bit immediate (in the li instruction) and allows multiple addresses to be accommodated within a single vector instruction executed in one clock cycle (dota).
- Circuit Simplicity and Component Count:** The 3-bit limit per address restricts the Register File to merely 8 registers ($2^3 = 8$), significantly simplifying the component's physical matrix. To support vector reads without inflating the hardware, only 2 main multiplexers and 2 small 3-bit adders are used to calculate consecutive registers (+1) in parallel. The Control Unit is equally straightforward: the opcode maps directly to the SinalALU, and RegWrite is continuously set to 1.
- Architecture Extensibility:** Although adding new core instructions appears constrained by the 2-bit opcode, the add, dot, and dota instructions retain zero padding (8, 8, and 5 unused bits, respectively). These free bits constitute a valuable strategic reserve. In the future, they can be repurposed as secondary selectors (acting as a substitute for func3) to implement subtractions or conditional jumps, thereby expanding the processor's capability without breaching the 16-bit limit.